

JSP3: Report for Task: WordNet Alignment

Vojtěch Mahdal & Dominik Fránek

Palacký University Olomouc

Faculty of Arts, Department of General Linguistics

vmahdal05@gmail.com, Dominik.franek@email.cz

February 25, 2026

1 Introduction

This report outlines the task of cross-lingual sentence alignment between Czech and English versions of Karel Čapek's *Válka s mloky* (War with the Newts). The goal is to synchronize parallel corpora at the sentence level to support further linguistic research, specifically Word Sense Disambiguation (WSD). The task involves handling structural translation discrepancies, such as many-to-one sentence mappings, and correcting for sequential drift using semantic embeddings.

2 Background

2.1 Resources

The primary data sources used for this project include:

- **Parallel JSON Corpora:** `twwtm-cs_human.json` (Czech) and `twwtm-en_human.json` (English).
- **Semantic Model:** The **LaBSE** (Language-Agnostic BERT Sentence Embedding) model, utilized via the `sentence-transformers` library to generate vector representations for cross-lingual comparison.
- **Existing Work:** Reference was made to the `lingtrain-aligner` framework for paragraph-level synchronization logic.

3 Approach

The alignment process follows a dynamic logic with drift correction. Below is the Python implementation used to extract sentences, encode them using LaBSE, and perform the sliding window alignment.

```
1 import json
2 import numpy as np
3 from sentence_transformers import SentenceTransformer
4 from sklearn.metrics.pairwise import cosine_similarity
5
6 # =====
7 # 1. UPGRADED EXTRACTION FUNCTION
8 # =====
9 def extract_sentences(filepath):
10     with open(filepath, 'r', encoding='utf-8') as f:
```

```

11     data = json.load(f)
12
13     sentences = []
14
15     for sent_id, sent_data in data.get('sent', {}).items():
16         if 'text' in sent_data:
17             sentences.append((sent_id, sent_data['text']))
18         else:
19             words = []
20             word_keys = sorted([k for k in sent_data.keys() if k.isdigit()],
key=int)
21             for key in word_keys:
22                 word_info = sent_data[key]
23                 # Prefer exact 'word' if it exists, otherwise use 'clemma' (
lemma)
24                 if 'word' in word_info:
25                     words.append(word_info['word'])
26                 elif 'clemma' in word_info:
27                     words.append(word_info['clemma'])
28
29             if words:
30                 sentences.append((sent_id, " ".join(words)))
31
32     sentences.sort(key=lambda x: int(x[0]))
33     return sentences
34
35 # --- Set your file paths here ---
36 CS_FILE_PATH = r"C:\Users\Mahdal\Desktop\data 2\twtn-cs-human.json"
37 EN_FILE_PATH = r"C:\Users\Mahdal\Desktop\data 2\twtn-en-human.json"
38
39 cs_data = extract_sentences(CS_FILE_PATH)
40 en_data = extract_sentences(EN_FILE_PATH)
41
42 cs_sentences = [item[1] for item in cs_data]
43 en_sentences = [item[1] for item in en_data]
44
45 # =====
46 # 2. LOAD SEMANTIC MODEL (LaBSE)
47 # =====
48 print("Loading LaBSE model...")
49 model = SentenceTransformer('sentence-transformers/LaBSE')
50
51 print("Encoding sentences (this may take a moment)...")
52 cs_embeddings = model.encode(cs_sentences, show_progress_bar=True)
53 en_embeddings = model.encode(en_sentences, show_progress_bar=True)
54
55 # =====
56 # 3. ALIGNMENT WITH LOOKAHEAD RE-ANCHORING
57 # =====
58 def align_with_drift_correction(src_sents, src_embs, tgt_sents, tgt_embs,
threshold=0.55):
59     aligned_pairs = []
60     unmatched_src = []
61     merged_emb_cache = {}
62
63     def get_merged_emb(text):
64         if text not in merged_emb_cache:
65             merged_emb_cache[text] = model.encode([text])[0]
66         return merged_emb_cache[text]
67
68     i, j = 0, 0
69     while i < len(src_sents) and j < len(tgt_sents):
70         sim_1_1 = cosine_similarity([src_embs[i]], [tgt_embs[j]])[0][0]

```

```

71     sim_1_2 = 0
72     if j + 1 < len(tgt_sents):
73         sim_1_2 = cosine_similarity([src_embs[i]], [get_merged_emb(
74             tgt_sents[j] + " " + tgt_sents[j+1])])[0][0]
75
76     sim_2_1 = 0
77     if i + 1 < len(src_sents):
78         sim_2_1 = cosine_similarity([get_merged_emb(src_sents[i] + " " +
79             src_sents[i+1])], [tgt_embs[j]])[0][0]
80
81     sim_skip_tgt = 0
82     if j + 1 < len(tgt_sents):
83         sim_skip_tgt = cosine_similarity([src_embs[i]], [tgt_embs[j+1]])
84         [0][0]
85
86     best_sim = max(sim_1_1, sim_1_2, sim_2_1, sim_skip_tgt)
87
88     if best_sim < threshold:
89         best_lookahead_sim = 0
90         best_i_jump, best_j_jump = 1, 1
91
92         for x in range(6):
93             for y in range(6):
94                 if x == 0 and y == 0: continue
95                 if i + x < len(src_sents) and j + y < len(tgt_sents):
96                     sim = cosine_similarity([src_embs[i+x]], [tgt_embs[j+y
97                     ]])[0][0]
98
99                     if sim > best_lookahead_sim:
100                         best_lookahead_sim = sim
101                         best_i_jump, best_j_jump = x, y
102
103         if best_lookahead_sim >= threshold:
104             for skipped_i in range(best_i_jump):
105                 unmatched_src.append(src_sents[i + skipped_i])
106             i += best_i_jump
107             j += best_j_jump
108         else:
109             unmatched_src.append(src_sents[i])
110             i += 1
111             j += 1
112
113     elif best_sim == sim_1_2:
114         aligned_pairs.append({'czech': src_sents[i], 'english': tgt_sents[j
115         ] + " " + tgt_sents[j+1], 'score': float(sim_1_2)})
116         i += 1; j += 2
117     elif best_sim == sim_2_1:
118         aligned_pairs.append({'czech': src_sents[i] + " " + src_sents[i+1],
119         'english': tgt_sents[j], 'score': float(sim_2_1)})
120         i += 2; j += 1
121     elif best_sim == sim_skip_tgt:
122         j += 1
123     else:
124         aligned_pairs.append({'czech': src_sents[i], 'english': tgt_sents[j
125         ], 'score': float(sim_1_1)})
126         i += 1; j += 1
127
128     return aligned_pairs, unmatched_src

```

Listing 1: Python implementation for LaBSE Alignment

4 Results

The algorithm successfully processed the parallel corpus:

- **Total Aligned Blocks:** 2,489.
- **Rejected Czech Sentences:** 353 (due to low semantic match).

5 Discussion

The high alignment rate (approx. 2,500 blocks) demonstrates that LaBSE is effective for mapping the complex literary style of Čapek. However, the rejection of 353 sentences suggests significant "drift" or liberal translation in certain segments. Future work should focus on lowering the rejection rate by integrating paragraph-level anchoring before sentence-level splitting.

6 Conclusions

We have implemented a functional pipeline for aligning Czech and English parallel texts. The resulting corpus is ready for integration into the NTU-MC or WSD-C projects. The next steps include getting `lingtrain-aligner` fully operational for expanded datasets.

7 Contributions

Regarding contributions, the code and methodology represent a collaborative effort resulting from profound research of the matter and extensive brainstorming of solutions to overcome technical bottlenecks. The workload included a comprehensive analysis of the source JSON files, the development of a custom regex-based pipeline to repair structural corruption, and the implementation of a semantic alignment strategy to mitigate sequential drift. The team collaborated closely on diagnosing the segmentation discrepancies inherent in human-translated texts and verifying the final interleaved output to ensure linguistic accuracy. The original report was drafted by Vojtěch Mahdal, with subsequent refinement and slight improvement propositions by Dominik Fránek.

8 Declaration

While the computational analysis, code development and the core research findings were developed independently by the authors, AI tools were utilized to optimize language clarity and grammar.

Table 1: Sample Alignment Output from LaBSE Model

Czech (CS)	English (EN)	Score
ANDRIAS SCHEUCHZERI	Andrias Sheuchzeri	0.84
ale kdybyste se zeptali na palubě lodi Kandong Bandoeng kapitána J . van Tocha , co to je tahle Tana Masa , před kterou právě zakotvil , nadával by chvíli a pak by vám řekl , že to je ta nejšpinavější díra v celých Sundských ostrovech , ještě mizernější než Tana Bala a přinejmenším tak zatracená jako Pini nebo Banjak ; že jediný s odpuštěním člověk , který tam žije - nepočítáme - li ovšem tyhle všivé Bataky - , je opilý obchodní agent , kříženec mezi Kubu a Portugalcem a ještě větší zloděj , pohan a prase než celý Kubu a celý běloch dohromady ;	If you looked up the little island of Tana Masa on the map you would find it just on the Equator, not far south of Sumatra; but if you were on the deck of the Kandong Bandoeng and asked its captain, J. van Toch, what he thought of this Tana Masa where you've just dropped anchor he would first curse for a short while and then he would tell you that it's the dirtiest hole all the Sunda Islands, even more loathsome than Tana Bala and easily as damnable as Pini or Banyak; that the only apology for a human being that lives there — not counting these louse-ridden Bataks, of course — is a drunken commercial agent, a cross between a Cuban and a Portuguese, and an even bigger thief, pagan and pig than the whole of Cuba and the whole of the white race put together; if there's anything in this world that's damnable then it's the damned life on this damned Tana Masa.	0.82
a že je - li na světě něco zatraceného , pak je to zatracený život na téhle zatracené Tana Masa , pane . Načež byste se ho patrně zeptali , proč tu tedy spustil ty zatracené kotvy , jako by tu chtěl užstat zatracené tři dny ;	And then, you might cautiously ask him why it is that he's just dropped his damned anchor as if he wanted to spend three damned days here; at which he would snort in irritation and grumble something about not being so damned stupid as to sail all the way to Kandong Bandoeng just to get this damned copra or palm oil, and there's nothing else here, but I've got my damned orders, and you will please be so kind as to mind your own damned business.	0.59
Ale kdybyste místo všetečných otázek nechali kapitána J . van Tocha , aby si bručel a proklínal pro sebe , mohli byste se dovědět víc .	But if, instead of asking all sorts of impertinent questions, you left Captain J. van Toch to grumble and curse by himself you might find out something more.	0.90
Jen ho nechte , jeho roztrpčenost si najde cestu sama .	Just leave him alone, he can sort out his foul mood by himself.	0.73